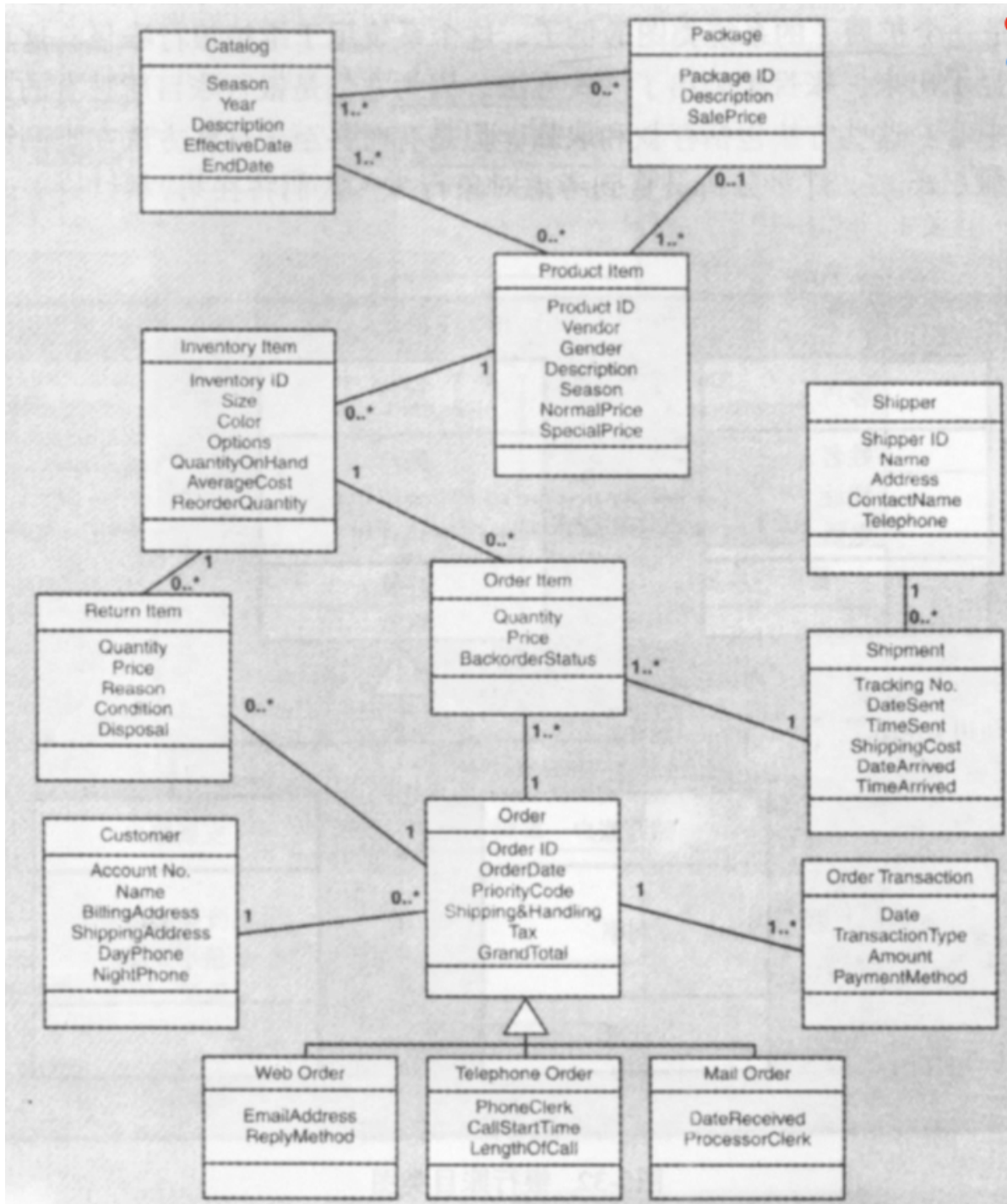


Chapter 10 需求建模：基于类的方法

Requirement Modeling: Class-Based Method



Class-Based Model包括的内容：

- Classes and Objects 类和对象
- Attributes 属性
- Operations 操作
- CRC Models 类-责任-协作模型
- Collaboration Diagrams 协作图
- Packages 包

※ 总的步骤:

1. 类——找到潜在的类（业务类） 10.1

检查在Chapter 9 中的需求模型中的一部分usage scenarios; 以UseCase为单位和依据, 对用例进行语法分析grammatical parse(每个UseCase检查有实义的名词或名词词组等)

2. 属性和操作——每个UseCase在类定义下来后, 初步定义类里面的属性和方法

3. CRC+聚合继承——建立CRC模型, 构建聚合和继承关系

4. 关联和依赖——构建类与类之间的关联关系、依赖关系

根据UseCase定义的类之间可能的关联 (包括同一个UseCase中的类和不同UseCase中的类)

5. 完整类图——构建完整的类图

第一次迭代通过语法扫描找到属性和方法

第二次迭代找到另一些属性方法补充

第三次迭代找到类之间协作关系

10.1 分析类 Analysis Classes

开始方法:

1. 检查在Chapter 9 中的需求模型中的一部分usage scenarios
2. 对用例进行语法分析grammatical parse(每个UseCase检查有实义的名词或名词词组等)

分析类的构成: 实体类

语法分析grammatical parse

由每个名词或名词短语确定并将其输入一个简单的表格。同义词要被记下来。如果需要类(名词)来实现解决方案, 那么它就是解决方案空间的一部分; 否则, 如果只有要描述解决方案所必需的类, 则它是问题空间的一部分。(例如, 将物品添加到购物车)

- 动词
 - 是否是实义动词
 - 是否归属于某个分析类 (可能是类的方法)
- 名词
 - 是否有实际意义 (没有就扔)
 - 如果有: 是否归属于某个类 (否: 作为类
 - 如果有: 作为类属性

分析类出现的方式:

注: 控件中的字段基本上都来自表中的字段

- **External Entity**外部实体——系统外
 - 其他的系统, 设备, 人; 比如传感器, 扫码器, 等等
- **Things in Info Domain**属于信息域内的东西——系统内
 - 比如: 报表, 展示, 信号灯
- **Occurrences or Events**上下文中出现的事件——事情

比如：机器人完成一系列活动（这些例子都怪怪的）

- **Roles** 角色（因为Role Class加一些属性可能是系统中的关键信息——人

比如：经理，工程师等

- **Organizational Units** 组织单元——组织

以group或team为单位

- **Places** 建立问题背景和系统的整体功能的地方——环境

比如：考试安排，则教室是一个分析类，SafeHome system中传感器的位置

- **Structures** 结构或用于定义一类对象或相关类的对象——结构

往往与聚合有关系，如汽车管理系统中的汽车，轮胎等（聚合关系）；用于定义一类对象或相关类的对象可能有 eg.control panel

※案例：SafeHome Security

以 SafeHome security 系统为例

⇒ 对需求规格中的文字描述进行语法扫描，找出名词/名词词组 ⇒ 确定类。

下面标蓝的可能是名词或名词词组

potential class of analysis class

1

The **SafeHome security function** enables the **homeowner** to **configure** the **security system** when it is **installed**, **monitors** all **sensors** connected to the **security system**, and **interacts** with the homeowner through the **Internet**, a **PC** or a **control panel**.

⇒ 也可能是 potential class
eg: 带有查询此硬件的有时

一定是 potential class

pass / 没有硬件定义

During installation, the **SafeHome PC** is used to **program** and **configure** the **system**. Each **sensor** is assigned a **number** and **type**, a **master password** is programmed for **arming** and **disarming** the system, and **telephone number(s)** are input for dialing when a **sensor event** occurs.

sensor no. type
security system 的密码

potential class

根据在系统内传感器的位置，受时间限制和限制等

含义的物体是 single (单物) 的名词 ⇒ 可能是其类名属性。

When a **sensor event** is **recognized**, the software **invokes** an **audible alarm** attached to the system. After a **delay time** that is **specified** by the homeowner during system configuration activities, the software **dials** a **telephone number** of a **monitoring service**, **provides** **information** about the **location**, **reporting** the nature of the **event** that has been detected. The **telephone number** will be **redialed** every **20 seconds** until **telephone connection** is **obtained**.

⇒ 是 potential class, 比如 security system 的 fault (故障)

是了 scope 之外我们就不控制，控制

视与 stakeholder 的 role 而定

可作为 potential class 也可不

The homeowner **receives** **security information** via a **control panel**, the **PC**, or a **browser**, collectively called an **interface**. The **interface** **displays** **prompting messages** and **system status information** on the **control panel**, the **PC**, or the

可能是一个 potential class

也可作为 system 或 sensor 的 panel

黑色：动词，蓝色：名词或名词词组

Potential Class	General Classification
homeowner	role or external entity
sensor	external entity
control panel	external entity
installation	occurrence
system (alias security system)	thing
number, type	not objects, attributes of sensor
master password	thing
telephone number	thing
sensor event	occurrence
audible alarm	external entity
monitoring service	organizational unit or external entity

这个List将一直做，直到已考虑处理叙述中的所有名词。请注意，我们调用列表中的每个条目“潜在”类。在最终决定之前，我们必须进一步考虑。

最终确定潜在类的6个方法（考试会考）

1. **Retained Info** 需要保留存在数据库中的信息——持久信息

比如：学生有学号信息等

2. **Needed Service** 必备的方法服务（用于修改类中属性值）——服务（方法）

比如：学生需要获取成绩

3. **Multiple Attributes** 潜在类应该有多个属性→低耦合——多属性

只有一个属性的潜在类往往并入到其他类中

4. **Common Attributes** 潜在类有通用属性（泛化，父子关系，基础关系）——通用属性

5. **Common Operations** 潜在类有通用方法——通用方法

6. **Essential Requirements** 潜在类（外部实体，必须的生成和消费信息）几乎总是被定义为需求模型中的类（无需care类似支付宝这样的外部实体类，因为我们不需要管理其中的数据）——需求相关

上面的6种特征要全部满足或几乎全部满足。不过提取这些类是主观的过程，之后的评估可能会导致类的增加减少

（请参阅英文课本P187页查看教员对类的深刻理解）

※以SafeHome Security继续为例子

练习: 找出潜在类

Potential Class

Characteristic Number That Applies

homeowner	rejected: 1, 2 fail even though 6 applies
sensor	accepted: all apply
control panel	accepted: all apply
installation	rejected
system (alias security function)	accepted: all apply
number, type	rejected: 3 fails, attributes of sensor
master password	rejected: 3 fails
telephone number	rejected: 3 fails
sensor event	accepted: all apply
audible alarm	accepted: 2, 3, 4, 5, 6 apply
monitoring service	rejected: 1, 2 fail even though 6 applies

还要注意: (可能考选择题, 和分析类有关)

1. 上述列表不是全包, 必须添加其他类以完成模型
2. 一些被拒绝的潜在类将成为被接受的那些类的属性 (例如, 数字和类型是传感器的属性, 主密码和电话号码可能成为系统的属性), 这是降级
3. 问题的不同陈述可能导致不同的“接受或拒绝”决定, 对use case的理解描述不同, 可能实现时有差别, 需要交流 (例如, 如果每个房主都有单独的密码或通过语音印刷识别, 房主课程将满足特征1和2, 并且已被接受)

10.2 细化属性Specify Attributes

分析类细化后形成设计类, 设计类包括: 实体类、控制类、边界类

步骤:

1. 进一步study用例, 一些名词词组或一些含数值信息的名词词组没有成为类, 可能是依附于某些类的属性
2. 没有在用例描述中出现的词, 通过data dict进行分解

以**UseCase图中的每个用例**作为分析单元, 研究每个用例

判断依据: 当前问题的上下文中有哪数据项

数据字典(输入输出数据的组织): 细分出以下作为System类的属性

identification information = system ID + verification phone number + system status

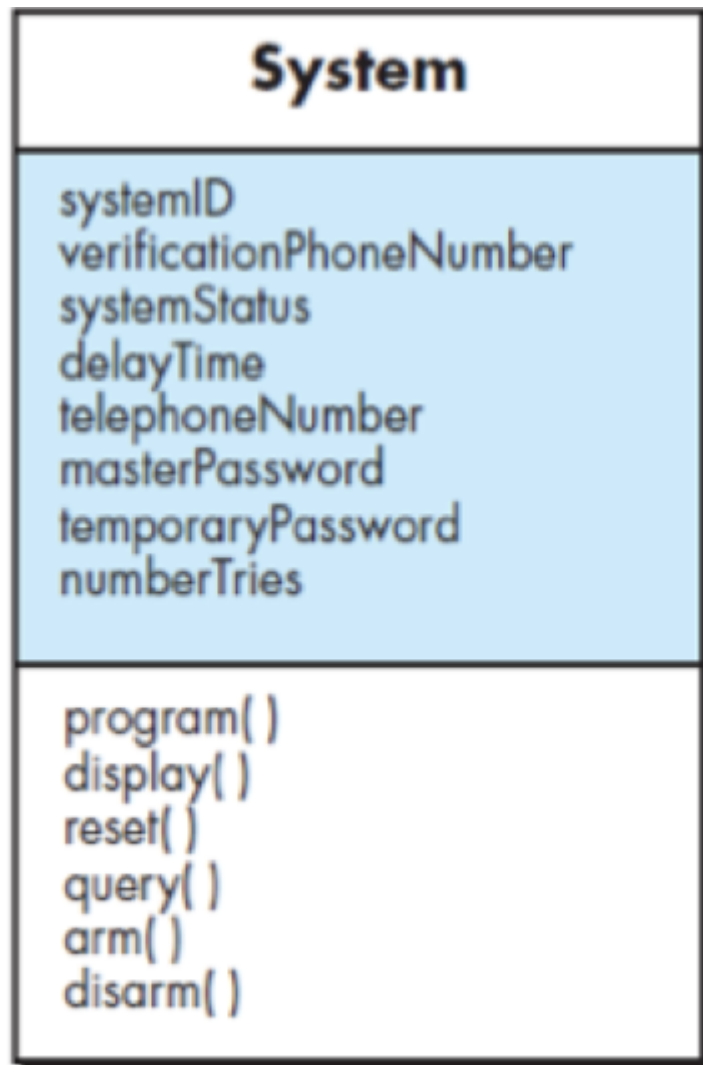
报警反应信息.
alarm response information = delay time + telephone number 反映时间+电话号码

激活/失效信息
activation/deactivation information = master password + number of allowable tries +
temporary password 密码错误的临时密码

界面输入时, 都是数据字典; 每个输入界面都要考虑数据字典,
根据输入的信息来决定空间, 比如报警—菜单、输入—一般空间

属性的定义——首先确定类需要做的事情
将这些事情转换为数据字典
数据字典中的元素(原数据)就是类里面的属性

一个分析类的例子：系统类



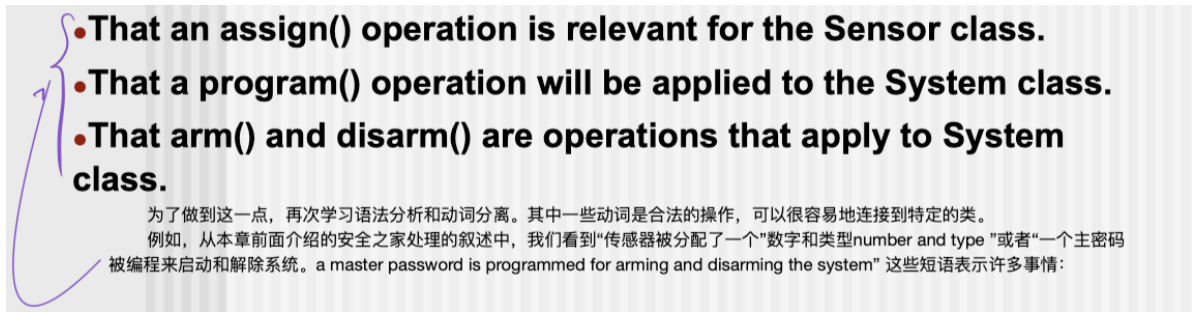
10.3 定义操作Operation（两类：内部计算/外部协作）

找Operation的方法：

- 语法解析：进一步扫描UseCase文字描述，找到实际意义的动词，补充类里面的方法

- 考虑对象之间发生的通信

例子：



- That an `assign()` operation is relevant for the **Sensor** class.
- That a `program()` operation will be applied to the **System** class.
- That `arm()` and `disarm()` are operations that apply to **System** class.

为了做到这一点，再次学习语法分析和动词分离。其中一些动词是合法的操作，可以很容易地连接到特定的类。
例如，从本章前面介绍的安全之家处理的叙述中，我们看到“传感器被分配了一个”数字和类型 `number and type` ”或者“一个主密码被编程来启动和解除系统。a master password is programmed for arming and disarming the system” 这些短语表示许多事情：

四个广泛类别：

1. 操作——操作以某种方式操纵数据（例如，添加，删除，重新格式化，选择）数据操作

operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting),

2. 计算——执行计算的操作

operations that perform a computation

3. 状态——询问对象状态的操作

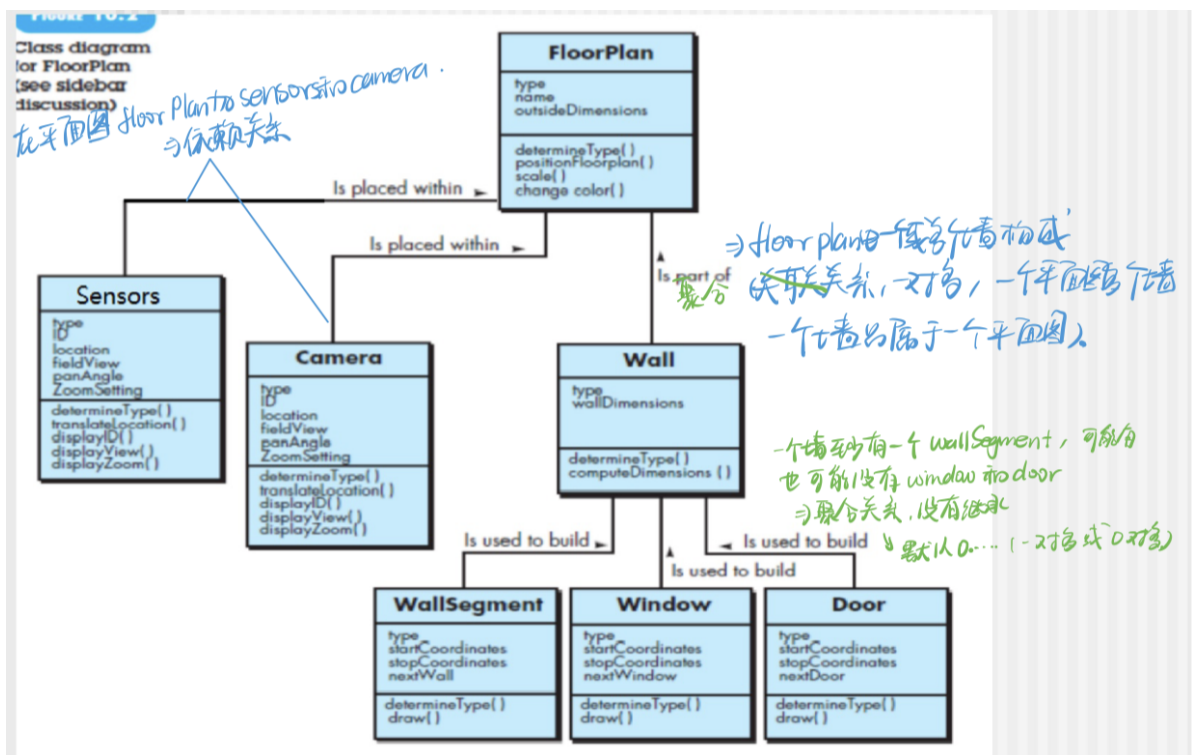
operations that inquire about the state of an object,

4. 监控——监视对象以发生控制事件的操作。

operations that monitor an object for the occurrence of a controlling event

进一步调查下，操作 `program()` 可能会被分成一个数字或更加详细的子操作以配置系统。比如 `program()` 暗示了具体的电话号码，配置系统的特征（e.g. 创建 `sensor table`，进入 `alarm` 特征，输入密码等，但目前我们先把 `program()` 细化成单一的方法）

例子：



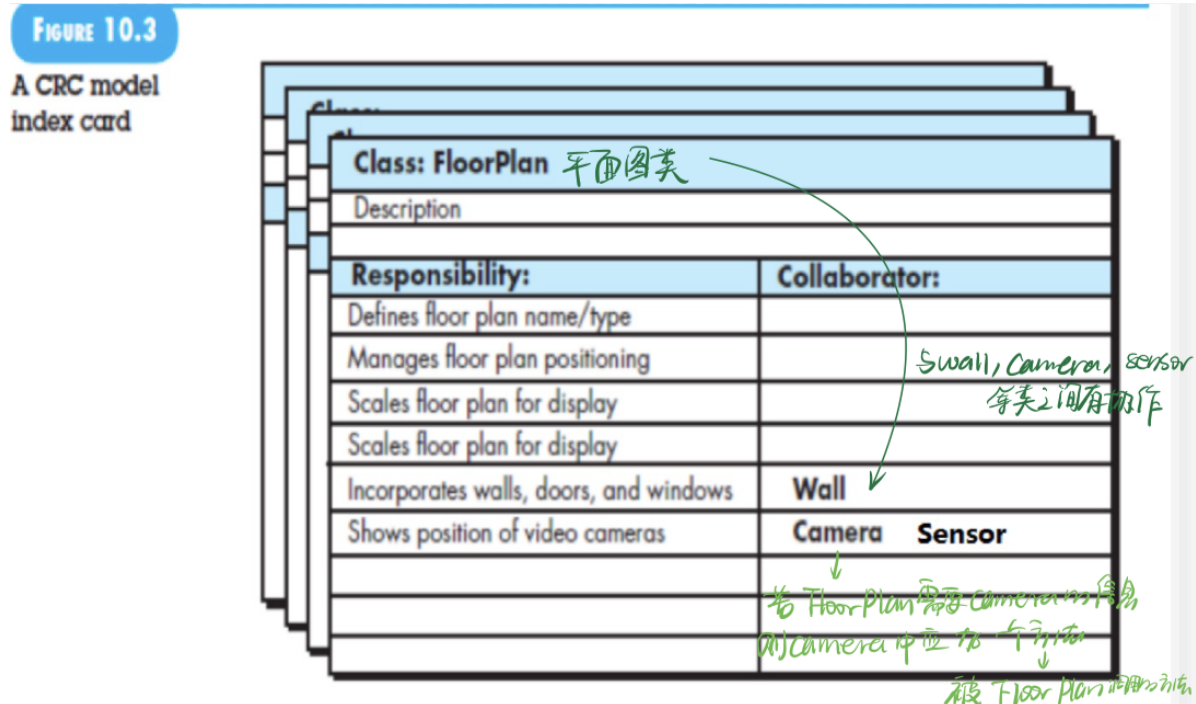
10.4 CRC建模Class-Responsibility-Collaborator(类的建模的基础上)

Responsibilities是与类相关的属性和操作

Collaborators是提供Class所需的其他Class, 其中包含填写Responsibility所需的信息。通常, 一个Collaboration意味着对某些动作的信息请求或请求。

collaborators are those classes that are required to provide a class with the information needed to complete a responsibility. In general, a collaboration implies either a request for information or a request for some action

例子:

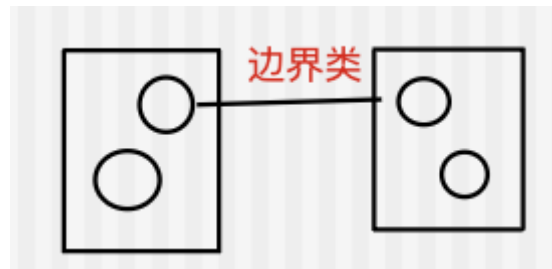


(TODO)

Class

Entity classes:业务实体类：通过对用例描述进行语法扫描得到的类为实体类

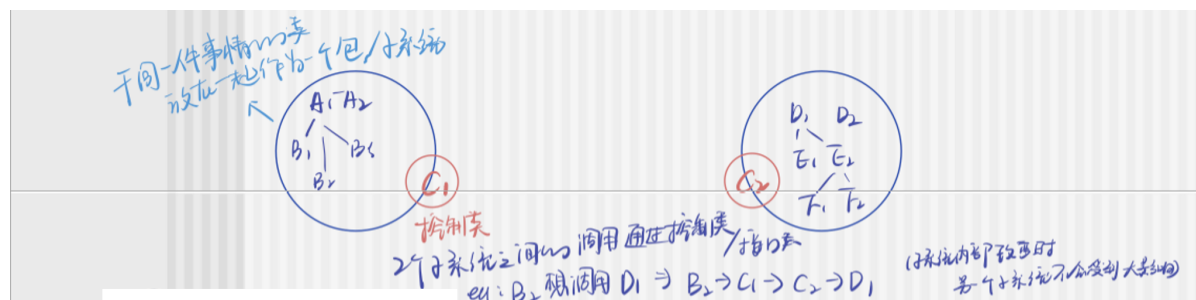
Boundary classes:边界类/界面类：用于创建接口的类，负责管理实体类对象与用户之间的交互（不会直接展现）



Controller(interface) class控制类：用于管理：

controller classes are not considered until the design activity has begun设计活动开始之前不会考虑控制类

1. **Creation&Updates** 实体对象的创建更新
2. **Boundary Instantiation** 边界类的实例化，用以获取实体类信息
3. **Communication** 管理一组对象之间的复杂通信
4. **Validation** 验证对象或用户和应用程序之间传送的数据



Responsibilities

attributes and operations

类的Responsibility进一步优化的五个指导思想：DLGES

1. **Distributed** 系统的功能应该被均匀的划分到不同的类的属性中去，每个类只获取并执行少量的系统功能任务信息，这样系统的内聚性（同时达到高内聚，什么类就做什么样的事情）得到加权
因此每个CRCcard都需要被review，确保没有extradinarily long list of responsibilities.
2. **General** 属性和方法需要具有一定的通用性
3. **Encapsulation** info和与它相关的behavior应该在同一类中。这实现了名为封装(encapsulation)的面向对象原则

Information and the behavior related to it should reside within the same class

4. **Localized** 一个东西的信息不应该被distributed而应该放在一个类中。一个Class应承担store和operate特定类型的信息的责任，实现了单一的功能
5. **Shared** 责任应在合适的情况下在相关类中共享

【往往出现在组合/聚合的类中】

e.g.游戏Sprite的身体：这些类中的每一个都有自己的属性（例如，位置，方向，颜色，速度），并且所有这些类都必须更新并显示，因为用户操纵操纵杆。因此，必须由每个对象共享职责更新和显示

Collaborations

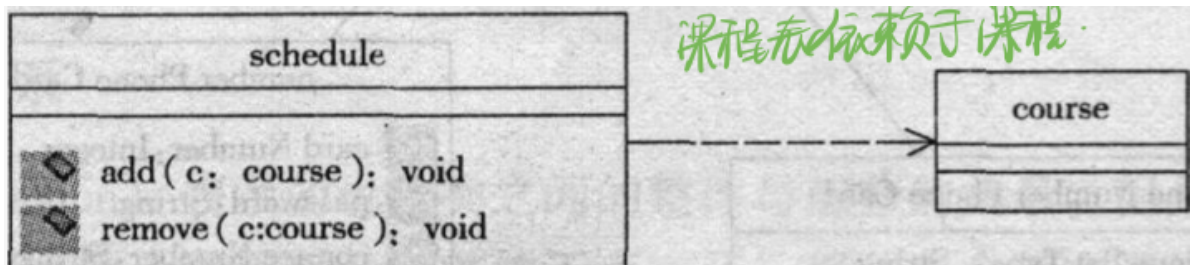
类以两种方式之一履行职责：

1. **Inner** 使用自己的方法来操纵自己的属性，从而履行特定的责任；
2. **Collaborate** 与其他类合作，给其他类用属性或使用其他类属性

(一个类没法自己实现完全功能时，合作)

类之间的关系：

- 部分关系 is-part-of relationship（整体与部分）
 - 组合
 - 聚合
- 关联关系 has-knowledge-of relationship（类之间想获取彼此的信息）
- 依赖关系 depend-upon relationship(一个类属性取决于另一个类属性，是一种特殊的关联关系)



当已开发完整的CRC模型时，stakeholder可以使用以下方法查看模型：（例如：用户登录的用例，其活动图为用户登录活动图.png）

1. **Seperation** 审核的所有人都发到CRC卡，有合作关系的卡分开，确保一个人手牌之间都没有协作关系
2. **Category** 所有的use-case scenario（以及相应的用例图）需要被分类
3. **Leader** 审查的领导人读取一个用例，进而引入一个已命名的对象。领导人将令牌token传递给有相应的类索引的人
4. **Describe** 当token传递时，要求卡片的持有者描述卡上注明的责任。审核组确定是否有一个（或更多）的责任满足用例要求
5. **Modify** 如果在索引卡上指出的责任和协作无法适应用例，则修改卡片。这可能包括新类（以及相应的CRC索引卡）的定义或在现有卡上的新或修订的责任或协作的规范。

10.5 关联和依赖Associations and Dependencies

FIGURE 10.4

A composite
aggregate
class

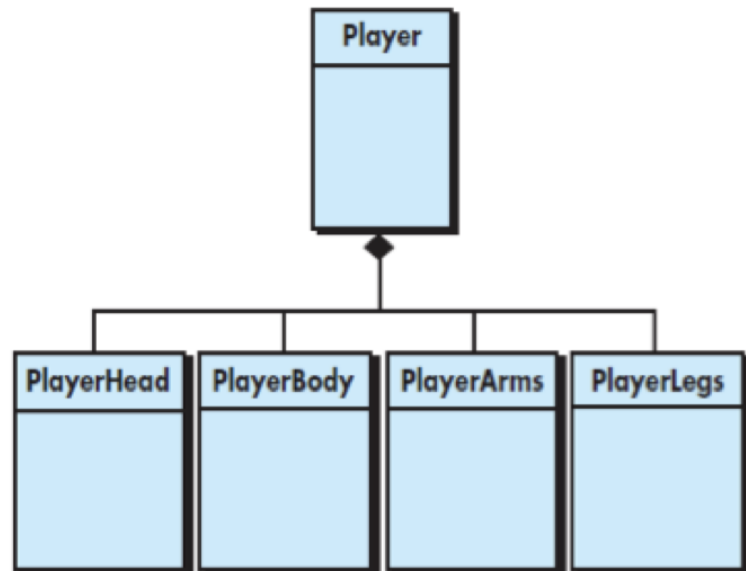


FIGURE 10.5

Multiplicity

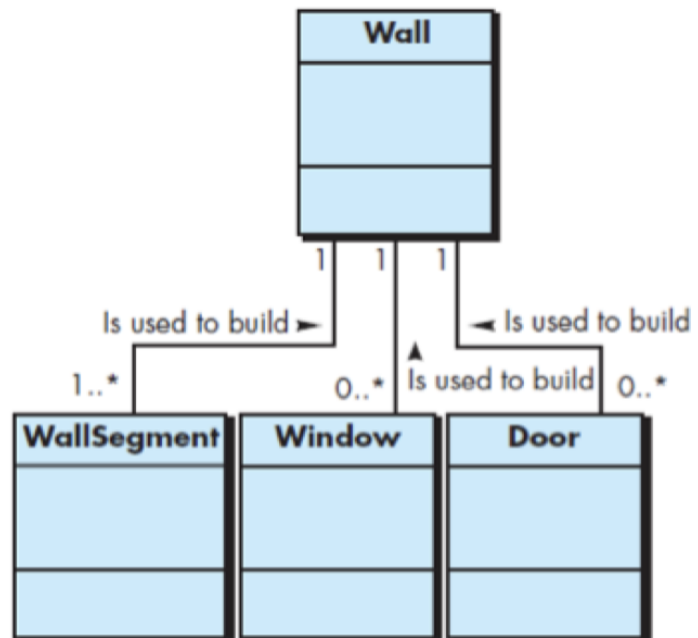
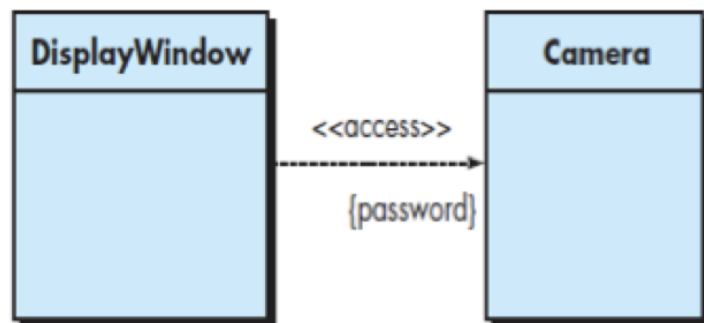


FIGURE 10.6

Dependencies



“one or more” is represented using 1..* , and “0 or more” by 0..*.

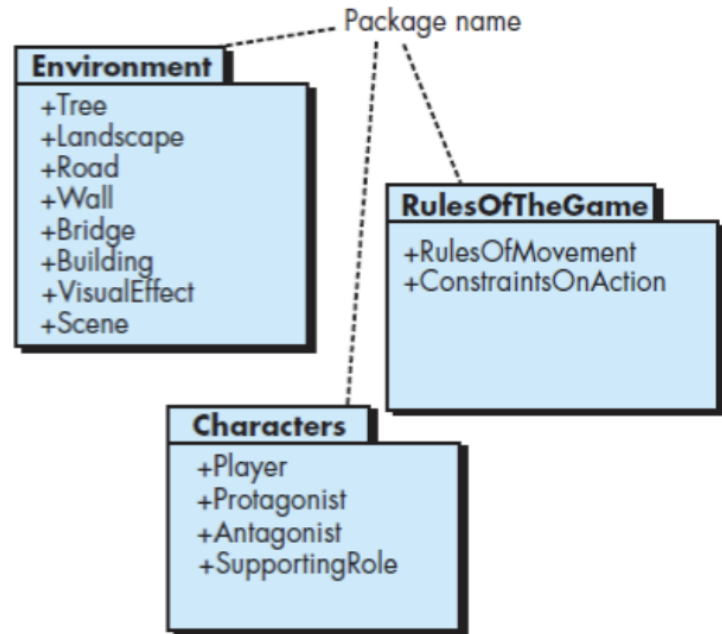
10.6 Analysis Packages (子系统)

类比较多的可以看做一个子系统，多个功能相似的也可以合并成一个

分析建模的重要部分是分类。也就是说，需求模型的各种元素（例如，用例，分析类别）被分类为将它们作为分组的分组包装的方式进行分类——就是分析包

FIGURE 10.7

Packages



Summary:

Class-based modeling uses information derived from use cases and other written application descriptions to identify analysis classes.